

核心模組與 API 參考手冊

本文件為 QUIC 媒體串流系統中「自訂撰寫」的 C++ 模組與類別提供完整的開發者指引。本系統在軟體架構上主要劃分為兩個核心模組：

1. 媒體處理與應用邏輯模組 (`include/app, src/app`)：負責視訊檔案解析、分包 (Packetization)、優先權串流調度 (Stream Scheduling) 以及無序重組 (Reassembly)。
2. 網路與 QUIC 傳輸 (`include/transport, src/transport`)：封裝了 Microsoft MsQuic 的實作，負責連線生命週期管理、Stream 建立、路徑 MTU 限制、串流優先權設定等。

傳輸協定與標頭格式

`moq::PacketHeader`

- 檔案路徑：`include/wire.hpp`
- 說明：一個 16-byte 結構體。每個透過 QUIC 傳送的應用層資料碎片前都會加上這個標頭。

```
#pragma pack(push, 1)
struct PacketHeader {
    uint32_t nalu_id;           ///< 全域嚴格遞增的 NALU 序號標識符。
    uint32_t payload_len;       ///< 緊跟在標頭後方的 Payload 位元組長度。
    uint16_t flags;             ///< 保留旗標，供未來擴充使用。
    uint16_t reserved;          ///< 對齊填充欄位或未來規劃使用。
    uint16_t fragment_idx;      ///< 目前碎片在該 NALU 中的索引。
    uint16_t fragment_count;    ///< 組成該完整 NALU 的碎片總數。
};
#pragma pack(pop)
```

媒體處理與應用邏輯模組

1. `moq::app::H265FileParser`

- 檔案路徑：`include/app/h265_parser.hpp`
- `00 00 00 01`

輔助結構體：`moq::app::Nalu`

- `const uint8_t* data`：指向 NALU 起始處（包含起始碼）的指標。
- `size_t size`：包含起始碼在內的 Nalu 總長度（位元組）。
- `size_t prefix_len`：起始碼的長度（3 位元組或 4 位元組）。
- `uint8_t get_nalu_type()`：提取並回傳 6-bit 的 H.265 NALU 類型。
- `bool is_important()`：判斷此 NALU 是否為影響解碼與同步的關鍵影格或中繼資料（例如：VPS type 32, SPS type 33, PPS type 34, IRAP/I-frame types 16–21）。

主要成員函式：

- `explicit H265FileParser(const std::string& filepath)`
 - 建構子，指定要解析的輸入視訊檔案路徑。

- `bool open()`
 - 開啟檔案、讀取至記憶體緩衝區並解析所有的 NALU 邊界。成功時回傳 `true`。
- `const std::vector<Nalu>& get_nalus() const`
 - 取得所有已解析 NALU 描述資訊的唯讀陣列參考。

2. `moq::app::DefaultPacketizer`

- 檔案路徑：`include/app/packetizer.hpp`
- 說明：繼承自 `moq::app::IPacketizer` 介面，負責將巨大的 NALU（如超過 MTU 的 I-frame）切割為適合網路傳輸的碎片封包，並為每個碎片加上 16-byte 的 `PacketHeader`。

輔助結構體：`moq::app::Packet`

- `moq::PacketHeader hdr`：預先填妥的應用層封包標頭。
- `std::vector<uint8_t> payload`：切片後的資料。

主要成員函式：

- `explicit DefaultPacketizer(size_t max_payload_size)`
 - 建構子，指定每個分片載荷的最大允許長度（位元組）。
- `std::vector<Packet> packetize(const Nalu& nalu, uint32_t nalu_id) override`
 - 實作分包邏輯。將 Raw Nalu 進行切片，填入對應的序號與索引，並回傳封包陣列。

3. `moq::app::Reassembler`

- 檔案路徑：`include/app/reassembler.hpp`
- 說明：伺服器端專用。由於 QUIC 多通道傳輸可能導致封包在網路上亂序抵達，此類別負責在記憶體中將分片依據 `fragment_idx` 進行快取與重組，並確保 NALU 按照原本的順序輸出給解碼器或寫入模組。

相關型態與回呼：

- `using WriteFn = std::function<void(const uint8_t* data, size_t len)>`
 - 當某個 NALU 的所有分片全數收齊並重組成功後，會觸發此寫入回呼。

主要成員函式：

- `explicit Reassembler(WriteFn writer)`
 - 建構子，綁定重組完成後的輸出寫入回呼。
- `void on_packet(const PacketHeader& hdr, const uint8_t* payload, size_t len)`
 - 接收來自傳輸層的碎片。利用內部的 `std::map` 將碎片暫存，當確認該 NALU 的所有碎片皆已到齊時，會觸發順序寫入邏輯。
- `uint32_t expected_id() const`
 - 回傳伺服器目前正在等待接收的下一個 NALU ID 序號。
- `size_t buffered_count() const`
 - 取得目前快取中（尚未收齊或亂序抵達）的 NALU 數量。

4. `moq::app::FileSink`

- 檔案路徑：`include/app/sinks.hpp`
- 說明：單純的輔助類別，管理二進位檔案的寫入。

主要成員函式：

- `explicit FileSink(const std::string& path)`
 - 建構子，開啟指定的檔案路徑準備寫入。
- `void write(const uint8_t* data, size_t len)`
 - 將二進位資料寫入至檔案串流快取中。
- `void flush()`
 - 強制將快取中的資料寫入實體磁碟。

5. `moq::app::Scheduler`

- 檔案路徑：`include/app/scheduler.hpp`
- 說明：用於多通道傳輸環境下，決定每個媒體 NALU 應該派發至哪一個 QUIC Stream。

主要成員函式：

- `explicit Scheduler(uint32_t num_streams)`
 - 建構子，初始化通道調度器，指定可用的 Stream 數量。
- `uint32_t pick_stream(uint32_t nalu_id) const`
 - 根據 NALU 屬性（如優先權分類或簡單的輪詢機制），回傳該 NALU 要使用的傳送 Stream 索引。

網路與 QUIC 傳輸

1. `moq::transport::QuicConfig`

- 檔案路徑：`include/transport/quic_transport.hpp`
- 說明：封裝連線初始化設定的結構體。

```
struct QuicConfig {
    std::string host;           ///< 目標伺服器 IP 位址或主機名稱。
    uint16_t port{0};          ///< 目標伺服器 UDP 埠號。
    uint32_t num_streams{1};    ///< 用戶端連線後預先建立的雙向 Stream 數量。
    std::string cert_file;      ///< 伺服器端 TLS 憑證路徑（僅伺服器模式需要）。
    std::string key_file;       ///< 伺服器端 TLS 私鑰路徑（僅伺服器模式需要）。
    uint16_t min_mtu{0};        ///< 自訂最小 Path MTU（選填，設 0 代表使用 MSQuic 預設 PMTUD）。
    uint16_t max_mtu{0};        ///< 自訂最大 Path MTU（選填，可以用來限制網路包大小）。
};
```

2. `moq::transport::MsQuicTransport`

- 檔案路徑：`include/transport/msquic_transport.hpp`
- 說明：封裝 Microsoft MsQuic API 的主要類別，將 C 語言的 Callback 架構轉化為執行緒安全的 C++ 物件。

共用 APIs：

- `MsQuicTransport()`
 - 建構子。載入 MsQuic API 函式指針表並完成初始化配置。
- `void run()`
 - 阻塞呼叫執行緒，讓伺服器/用戶端保持運作，直至連線關閉。
- `void set_rcv_callback(PacketRcvCb cb)`

- 註冊接收回呼。當底層接收並完成 framing 重組出完整的 `PacketHeader` 與 `Payload` 時，實主動呼叫此回呼。

用戶端 APIs：

- `bool start_client(const QuicConfig& cfg)`
 - 向目標伺服器發起連線與 TLS 握手，連線成功後自動開啟指定數量的 Streams。
- `bool wait_connected()`
 - 阻塞式等待，直到 TLS 握手完成且連線確立。
- `bool wait_stream_ready()`
 - 阻塞式等待，直到用戶端的所有發送通道皆已開啟完畢。
- `bool set_stream_priority(uint32_t stream_index, uint16_t priority)`
 - 設定特定通道的優先權（例如將傳輸 I-frame 的 Stream 0 設為最高的 `0xFFFF`，其他設為 `0x0001`）。
- `bool send(uint32_t stream_index, const moq::PacketHeader& hdr, const uint8_t* payload)`
 - 將標頭與載荷拷貝包裝至 `SendContext` 中，並調用非同步的 `StreamSend` 發送。
- `void set_send_expected(uint32_t n)`
 - 設定本次發送任務預期要成功傳送（收到對端 ACK）的總封包數。
- `bool wait_all_sends_done(uint32_t timeout_ms = 5000)`
 - 阻塞等待，直到預期數量的封包皆已觸發 `SEND_COMPLETE` 回呼。
- `void graceful_shutdown()`
 - 對所有開啟中的 Stream 發起關閉訊號。
- `bool wait_stream_shutdown(uint32_t timeout_ms)`
 - 阻塞等待，直到所有通道安全結束與對端的 ACK 確認關閉。

伺服器端 APIs：

- `bool start_server(const QuicConfig& cfg)`
 - 在指定埠號開啟伺服器監聽，並載入憑證以準備接收連線。

簡易範例程式碼

以下範例展示如何將上述模組組合起來：在用戶端讀取並解析 H.265 檔案、將影格切片、透過不同優先權的 QUIC Streams 發送出去，最後在伺服器端進行重組並寫入檔案。

用戶端傳送邏輯

```

#include "app/h265_parser.hpp"
#include "app/packetizer.hpp"
#include "transport/msquic_transport.hpp"
#include <iostream>

int main() {
    // 1. 解析 H.265 原始影片檔以取得 NALU 列表
    moq::app::H265FileParser parser("test_input.h265");
    if (!parser.open()) return -1;
    const auto& nalus = parser.get_nalus();

    // 2. 配置並啟動 QUIC 傳輸層用戶端
    moq::transport::QuicConfig cfg;
    cfg.host = "127.0.0.1";
    cfg.port = 4433;
    cfg.num_streams = 2; // Stream 0 (重要影格), Stream 1 (次要影格)

    moq::transport::MsQuicTransport quic;
    if (!quic.start_client(cfg) || !quic.wait_connected() || !quic.wait_stream_ready()) {
        return -1;
    }

    // 設定通道優先權: Stream 0 為最高 (0xFFFF), Stream 1 為最低 (0x0001)
    quic.set_stream_priority(0, 0xFFFF);
    quic.set_stream_priority(1, 0x0001);

    // 3. 初始化分包器 (設定每個分片載荷上限為 1200 bytes)
    moq::app::DefaultPacketizer packetizer(1200);

    // 4. 開始發送分片封包
    for (size_t i = 0; i < nalus.size(); ++i) {
        // 重要影格 (I-frame/VPS/SPS/PPS) 走 Stream 0, 次要影格 (P-frame) 走 Stream 1
        uint32_t stream_idx = nalus[i].is_important() ? 0 : 1;

        auto fragments = packetizer.packetize(nalus[i], static_cast<uint32_t>(i));
        for (const auto& fragment : fragments) {
            quic.send(stream_idx, fragment.hdr, fragment.payload.data());
        }
    }

    // 5. 關閉連線並等待通道關閉 ACK
    quic.graceful_shutdown();
    quic.wait_stream_shutdown(5000);
    return 0;
}

```

伺服器端接收與重組邏輯

```

#include "app/reassembler.hpp"
#include "app/sinks.hpp"
#include "transport/msquic_transport.hpp"
#include <iostream>

int main() {
    // 1. 設定檔案輸出與重組器
    moq::app::FileSink sink("received_output.h265");
    moq::app::Reassembler reasm([&](const uint8_t* data, size_t len) {
        sink.write(data, len); // 當 NALU 完整到齊時，依序寫入檔案
    });

    // 2. 配置並啟動 QUIC 傳輸層伺服器
    moq::transport::QuicConfig cfg;
    cfg.host = "0.0.0.0";
    cfg.port = 4433;
    cfg.cert_file = "server.crt";
    cfg.key_file = "server.key";

    moq::transport::MsQuicTransport quic;
    quic.set_rcv_callback([&](const moq::PacketHeader& hdr, const uint8_t* payload, size_t len) {
        reasm.on_packet(hdr, payload, len); // 將收到的碎片丟給重組器
    });

    if (!quic.start_server(cfg)) return -1;

    // 3. 阻塞主執行緒以維持伺服器監聽運作
    quic.run();
    sink.flush();
    return 0;
}

```